SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Mohan Kumar J	**Reg. No.:**	192424060
Section / Batch:	2024 [AI&DS]	**Date:**	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Healthcare Resource Scheduling Optimization Using Greedy Scheduling

1. Introduction

Hospitals need to manage many patients, doctors, nurses, operating theatres, and medical equipment. Since these resources are limited, proper scheduling is very important.

In a hospital, several patients may be waiting for surgery at the same time. Some surgeries may be emergency cases, while others may be routine or elective. If surgeries are not scheduled properly, patients may have to wait longer, operating theatres may remain unused, and staff or equipment may be overbooked.

The main objective of this project is to develop a **Greedy Scheduling Algorithm** for assigning surgeries to operating theatres efficiently.

The proposed system aims to:

- Reduce patient waiting time.
- Give priority to emergency cases.
- Use operating theatres efficiently.
- Avoid staff conflicts.
- Avoid equipment conflicts.
- Reduce unnecessary idle time.
- Handle new emergency cases quickly.

2. Healthcare Scheduling Problem and Constraints

The hospital must decide which surgery should be performed first, when it should be performed, and which resources should be assigned to it.

Each surgery may have:

- Patient arrival time
- Surgery priority
- Surgery duration
- Surgeon
- Nursing staff
- Anesthetist
- Required equipment
- Deadline

Major Constraints

1. Emergency cases:
   Emergency surgeries must receive higher priority because delays can affect patient safety.
2. Operating theatres:

   A single theatre cannot be used by two surgeries at the same time.
3. Staff availability:

   A surgeon, anesthetist, or nurse cannot be assigned to two surgeries at the same time.
4. Equipment availability:

   Special equipment must be available before a surgery can start.
5. Surgery duration:

   The scheduler must consider how long each surgery will take.
6. Patient arrival:

   A surgery cannot start before the patient is available.

$$\textbf{Start Time} \geq \textbf{Arrival Time}$$

7. Working hours:

   The schedule should follow the working hours of staff and operating theatres.

Therefore, the scheduling problem is a resource allocation and scheduling problem with multiple constraints.

3. Greedy Scheduling Approach

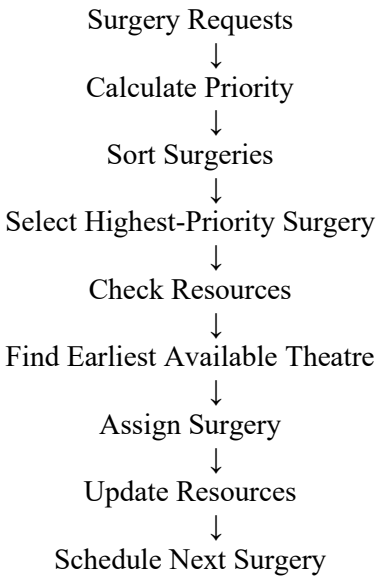
A **Greedy Algorithm** makes the best available decision at each step instead of checking every possible schedule.

For this healthcare problem, the algorithm selects the most important feasible surgery and assigns it to the earliest available suitable operating theatre.

The surgeries are considered using the following order:

1. Emergency priority
2. Patient waiting time
3. Deadline
4. Surgery duration
5. Arrival time

The basic idea is:



This approach is useful because hospitals often need quick scheduling decisions, especially when emergency cases arrive.

4. Proposed Algorithm and Implementation

Greedy Scheduling Algorithm

Input:
- Surgeries
- Operating Theatres
- Staff
- Equipment

Sort surgeries by:
- Priority
- Waiting time
- Deadline
- Duration
- Arrival time

For each surgery:

- Check available theatres
- Check surgeon availability
- Check staff availability
- Check equipment availability

- Find the earliest feasible start time

If resources are available:
Assign the surgery
Calculate end time
Update all resources

Otherwise:
Put the surgery in the waiting queue

Return the final schedule

Simple Python Implementation

```
from dataclasses import dataclass


@dataclass
class Surgery:
    surgery_id: str
    patient: str
    priority: int
    arrival: int
    duration: int
    deadline: int
    surgeon: str


@dataclass
class Theatre:
    theatre_id: str
    available_from: int


def greedy_schedule(surgeries, theatres, surgeon_available):

    # Sort by priority, arrival time, deadline and duration
    surgeries.sort(
        key=lambda x: (
            -x.priority,
            x.arrival,
            x.deadline,
            x.duration
        )
    )

    schedule = []

    for surgery in surgeries:

        best_theatre = None
        best_start = float("inf")
```

```
        for theatre in theatres:

            start_time = max(
                surgery.arrival,
                theatre.available_from,
                surgeon_available[surgery.surgeon]
            )

            if start_time < best_start:
                best_start = start_time
                best_theatre = theatre

        end_time = best_start + surgery.duration

        schedule.append({
            "Surgery": surgery.surgery_id,
            "Patient": surgery.patient,
            "Theatre": best_theatre.theatre_id,
            "Start": best_start,
            "End": end_time,
            "Waiting": best_start - surgery.arrival
        })

        best_theatre.available_from = end_time
        surgeon_available[surgery.surgeon] = end_time

    return schedule
```

The program sorts surgeries according to priority and then assigns each surgery to the earliest available theatre while considering surgeon availability.

5. Example and Scheduling Result

Consider a hospital with two operating theatres and five surgeries.

Surgery	Priority	Arrival	Duration	Surgeon
S001	5	0	90 min	Dr. A
S002	3	10	120 min	Dr. B
S003	5	20	60 min	Dr. A
S004	2	30	180 min	Dr. C
S005	4	40	90 min	Dr. B

Here, a higher priority number means a more urgent surgery.

The algorithm first considers S001 and S003 because both have the highest priority. However, both require Dr. A, so they cannot be performed at the same time.

An example schedule is:

Surgery	Theatre	Start	End	Waiting Time
S001	OT-1	0	90	0
S003	OT-2	90	150	70
S002	OT-1	90	210	80
S005	OT-2	150	240	110
S004	OT-1	210	390	180

The waiting time is calculated as:

$$\mathbf{WaitingTime = StartTime - ArrivalTime}$$

This example shows that the algorithm considers both **surgery priority and resource availability**.

If an emergency surgery arrives, the algorithm checks the available resources and schedules it at the earliest feasible time.

6. Performance Analysis

The performance of the proposed greedy scheduling algorithm can be evaluated using several important metrics. These metrics help determine whether the algorithm reduces patient waiting time, improves operating theatre utilization, handles emergency cases efficiently, and uses computational resources effectively.

6.1 Average Waiting Time

Average waiting time measures the average amount of time patients wait before their surgery begins.

6.1 Time Complexity

Let:

- (n) = number of surgeries
- (m) = number of operating theatres

The algorithm first sorts the surgeries based on priority, waiting time, deadline, and duration. Sorting requires:

O(nlog n)

After sorting, the algorithm checks the available operating theatres for each surgery. This requires:

O(nm)

Therefore, the overall time complexity is:

O(nlog n + nm)

This complexity is suitable for practical hospital scheduling because the algorithm can generate schedules relatively quickly, even when the number of surgeries increases.

6.2 Space Complexity

The algorithm stores information about surgeries, operating theatres, staff availability, and the generated schedule.

The main schedule contains information for each surgery, so the space required for the schedule is approximately:

O(n)

Therefore, the algorithm has a practical space requirement and can efficiently manage the scheduling information for a large number of surgeries.

7. Validation and Testing

Validation is important because an incorrect hospital schedule can cause operational and safety problems.

The following conditions must be checked.

Theatre Validation

Two surgeries must not use the same theatre at the same time.

Surgeon Validation

A surgeon must not be assigned to overlapping surgeries.

Staff Validation

An anesthetist or nurse must not be assigned to two simultaneous surgeries when the resource is exclusive.

Equipment Validation

Required equipment must be available during the complete surgery.

Patient Arrival Validation

$$\textbf{Start Time} \geq \textbf{Arrival Time}$$

Emergency Validation

Emergency surgeries should receive higher priority than normal surgeries when resources are available.

Test Cases

Test Case	Situation	Expected Result
TC01	One surgery	Surgery is scheduled
TC02	Multiple surgeries	Priority order is followed
TC03	Same surgeon	No overlapping surgeries
TC04	Emergency surgery	Emergency receives priority
TC05	Theatre unavailable	Another theatre is selected
TC06	Equipment unavailable	Surgery waits
TC07	All theatres occupied	Surgery enters waiting queue

These test cases help confirm that the algorithm produces a valid and reliable schedule.

8. Healthcare Management Interpretation

The proposed system can help hospital managers make better resource allocation decisions.

The system can provide information about:

- Patient waiting time
- Emergency response time
- Theatre utilization
- Staff utilization

- Equipment usage
- Delayed surgeries
- Overtime
- Resource bottlenecks

For example, suppose a hospital has three operating theatres but only one anesthetist is available. Even though the theatres are free, surgeries may still be delayed. In this situation, the anesthetist is a **bottleneck resource**.

Management can use this information to:

- Allocate additional staff
- Change staff shifts
- Improve theatre planning
- Purchase additional equipment
- Reserve theatre capacity for emergency cases

Therefore, the algorithm can support both **daily scheduling and long-term hospital resource planning**.

9. Advantages and Limitations

Advantages

1. Fast Scheduling

The greedy algorithm can make scheduling decisions quickly.

2. Emergency Prioritization

Emergency surgeries can be scheduled before routine surgeries.

3. Reduced Waiting Time

Patients who have waited longer can receive higher priority.

4. Better Theatre Utilization

Available operating theatres can be used more efficiently.

5. Resource Awareness

The system can consider staff, surgeons, theatres, and equipment.

6. Real-Time Adaptability

New emergency cases can be added to the existing schedule.

7. Simple Implementation

The algorithm can be implemented using basic programming techniques such as sorting and priority queues.

Limitations

1. No Guaranteed Optimal Solution

A greedy algorithm selects the best current option but may not produce the globally best schedule.

2. Complex Hospital Constraints

Real hospitals may have many additional constraints that require advanced optimization methods.

3. Frequent Emergency Cases

Continuous emergency cases may require the schedule to be updated frequently.

4. Staff Fatigue

Maximum resource utilization should not be allowed to create excessive staff workload.

5. Clinical Judgment

The algorithm should support hospital staff and doctors rather than replace clinical decision-making.

Starvation Prevention

If emergency surgeries continuously arrive, elective patients may wait for too long. This can be reduced using **aging**.

$$\textbf{EffectivePriority} = \textbf{OriginalPriority} + \textbf{AgingFactor WaitingTime}$$

As waiting time increases, the patient's priority also increases. This creates a better balance between emergency cases and elective patients.

10. Conclusion

Healthcare resource scheduling is a challenging problem because hospitals have limited operating theatres, medical staff, equipment, and time. At the same time, hospitals must handle both emergency and elective surgeries.

The proposed **Greedy Healthcare Resource Scheduling Algorithm** provides a simple and efficient solution. It considers surgery priority, patient waiting time, deadlines, duration, and resource availability before assigning a surgery.

The algorithm selects the most appropriate feasible surgery and assigns it to the earliest available operating theatre. It also checks surgeon, staff, and equipment availability to avoid conflicts.

The performance can be evaluated using:

- Average patient waiting time
- Operating theatre utilization
- Emergency response time
- Deadline compliance
- Staff utilization
- Computational complexity

The time complexity of the proposed algorithm is:

$$\mathbf{O(nn + nm)}$$

which allows the system to generate schedules efficiently.

Although the greedy approach does not always guarantee the globally optimal schedule, it provides a good balance between speed, simplicity, patient priority, and resource utilization.

Overall, the proposed solution demonstrates how a greedy scheduling technique can be applied to a real healthcare management problem. It can help hospitals reduce patient waiting time, improve operating theatre utilization, handle emergency cases efficiently, and make better resource allocation decisions while keeping patient safety and clinical judgment as the highest priorities.

SIMATS Online Lab

assactivity/attempt?assignment_id=1209

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

MOHAN KUMAR
192424060RA

```python
import sys
def solve_scheduling():
    """
    Function to solve the Healthcare Resource Scheduling problem
    using a Greedy Approach.
    """
    surgeries = []
    print("--- Healthcare Resource Scheduling System ---")
    print("Enter surgery details. Format: ID OT_NAME START_TIME FINISH_
    print("Type 'DONE' when you have finished entering all records.\n")
    while True:
        try:
            line = input().strip()
            if line.upper() == 'DONE' or not line:
                break
            parts = line.split()
            if len(parts) == 4:
                s_id = parts[0]
                ot_name = parts[1]
                start_time = int(parts[2])
                finish_time = int(parts[3])
                surgeries.append({
                    'id': s_id,
                    'ot': ot_name,
                    'start': start_time,
                    'finish': finish_time
                })
            else:
                print(f"Invalid input format: {line}. Please try again.
                continue
        except EOFError:
            break
        except ValueError:
            print("Error: Start and Finish times must be integers.")
            continue
    if not surgeries:
        print("No surgery data provided.")
        return
    surgeries.sort(key=lambda x: x['finish'])
    ot_availability = {}
    scheduled_surgeries = []
    for s in surgeries:
        s_id = s['id']
        ot = s['ot']
        s_start = s['start']
        s_finish = s['finish']
        if ot not in ot_availability or ot_availability[ot] <= s_start:
            scheduled_surgeries.append(s)
            ot_availability[ot] = s_finish
        else:
            pass
    print("\nOutput Greedy Schedule (surgery_id, OT, start, finish):")
    for item in scheduled_surgeries:
        print(f"{item['id']} {item['ot']} start {item['start']} finish
def display_header():
    """
    Displays a professional header for the management system.
    """
    print("="*50)
    print("SIMATS HEALTHCARE RESOURCE MANAGEMENT SYSTEM")
    print("Optimization Module: Greedy Interval Scheduling")
    print("="*50)
if __name__ == "__main__":
    display_header()
    solve_scheduling()
    print("\n" + "="*50)
    print("Scheduling Task Completed Successfully.")
    print("="*50)
```

```
S1 OT1 0 60
S2 OT1 30 90
S3 OT2 15 75
S4 OT2 75 120
```

Output

```
==================================================
SIMATS HEALTHCARE RESOURCE MANAGEMENT SYSTEM
Optimization Module: Greedy Interval Scheduling
```